



UNIVERSITY OF LEEDS

COMP1850 Programming

Week 1 Session 1

Today's Aims:

Outline the languages and tools for this course

- Become familiar with the structure and staff of the course
- Understand how you will be assessed this year
- Understand the tools which you will use this year for programming
- Feel confident using them



Ground Rules

Respect other learners – keep chat quiet and sensible

Engage with the sessions!

Discussion

Which programming languages have you used?

- What are their properties?
- Find another similar language

What tools do you use to program?

- Operating system
- Development environment
- Version control
- What are the advantages and disadvantages?

The Course

- An introduction to programming:
 - High level programming with Python
 - Low level programming with C
 - Relational database programming with SQL
- Industry standard tools:
 - Git
 - VSCode / Codespaces (linux)
- Programming skills:
 - Problem solving
 - Creative thinking
 - Debugging and refactoring

Why these languages?

- Python is currently the most popular language
 - A general purpose programming language (for learners)
 - A framework language for AI and other areas (for professionals)
- SQL is a domain-specific language for database access
 - Relational databases are ubiquitous in desktop and mobile applications
- C underpins many applications and platforms
 - Linux, Windows and Mac O/S are primarily written in C
 - The Python language interpreter is written in C

When you finish this module....

You should be able to:

- Practically
 - Read and write code in C, Python and SQL
 - Write programs from scratch given a specification
 - Work on someone else's code - adding to it or improving it
- Conceptually
 - Understand the differences between languages and their uses
 - **Think like a programmer**

Module Structure

- **Two teaching sessions a week**
 - interactive lab sessions where you learn and practice programming
- Independent study
 - you are expected to spend a few hours per week finishing work from the labs, researching, and developing your skills.
- Office hours
 - a chance to go over something you don't understand, ask questions, and get a bit of 1-on-1 support with a lecturer
- Weekly worksheet
 - independent work which will add to your assessment portfolio

Module Assessment

- Weekly worksheet
 - Independent work which will add to your assessment portfolio
 - You must complete and pass every worksheet to pass the module
- Final assessment
 - A written exam at the end of the year
 - The final exam allows you to receive a higher mark.

Assessment Portfolio

- One worksheet activity a week will be labelled as a **portfolio task**
- You **must** complete and submit this onto Gradescope or Minerva as required
- This creates a portfolio of evidence showing that you meet the criteria to progress to level 2
- You need to pass **every portfolio task** to progress.

We strongly recommend setting yourself **1-2 hours per week** aside to **complete your portfolio task** – making a routine early helps you to keep **organised through the year!**

Independent Study

For each module, you are expected to spend **about 8 hours** working independently on your studies per week.

This isn't limited to just going over notes or doing your worksheet – this could also include:

- Doing some programming on your own project
- Looking at programming challenges
- Reading or watching other resources such as books, guides or tutorials
- Researching other topics which you're interested in
- Participating in a hackathon or programming contest.

These Sessions

We will introduce the topic of the session and go through slides, examples, and some activities as a group.

In each session there will be teaching assistants as well as a lecturer – use all of us! Ask questions, get support, and let us know if there's something you're particularly interested in.

Unlike other modules, these sessions are not recorded - if you miss a session, you will need to catch up in your own time.



Questions!

Do you have any questions about the module structure or assessment?

Version Control & Git

Keeping track of your code



In industry...



- You are always working in a team
- You work on the same codebase as other people
- The codebase may be **live** (already deployed and running)



In university...

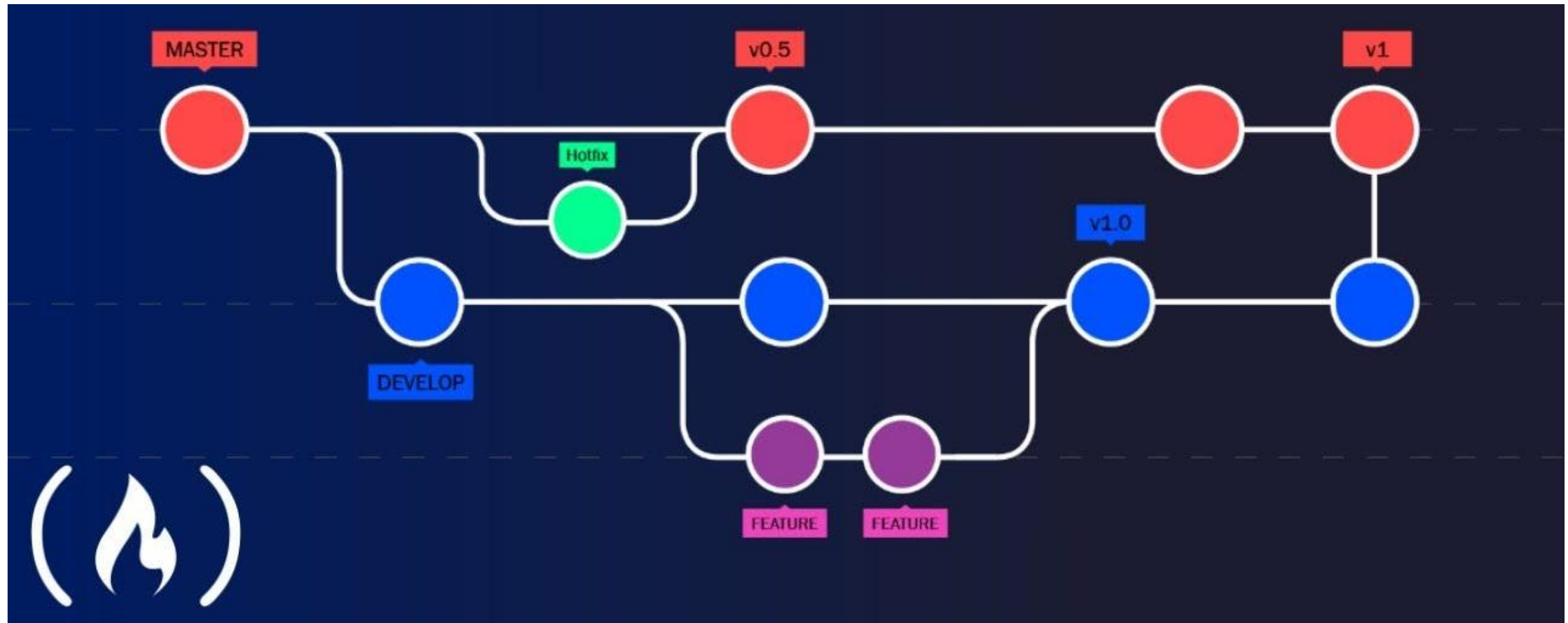
- Working individually or in teams
- You may be working on university machines or your own
- Often creating new code
- You need a secure backup of your work
- You will break things and want to recover working code!

Version Control

Version control systems:

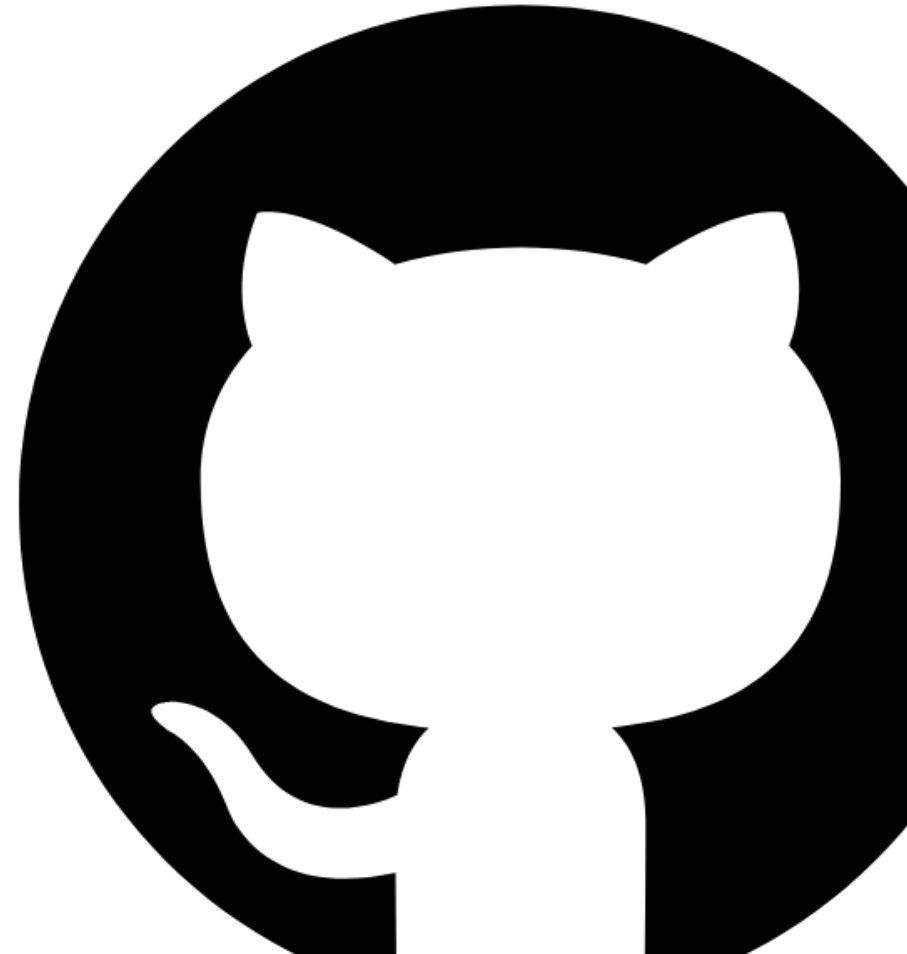
- allow programmers to work concurrently on the same code.
- allow you to track the history of development.
- allow you to restore previous versions.

Version Control development tree



Github

- A git-based platform for version control
- Also has lots of cool open source projects!
- We're going to be using it for **everything**
 - throughout your degree
- Provides a very useful integrated development environment: **Codespaces**



Task 1

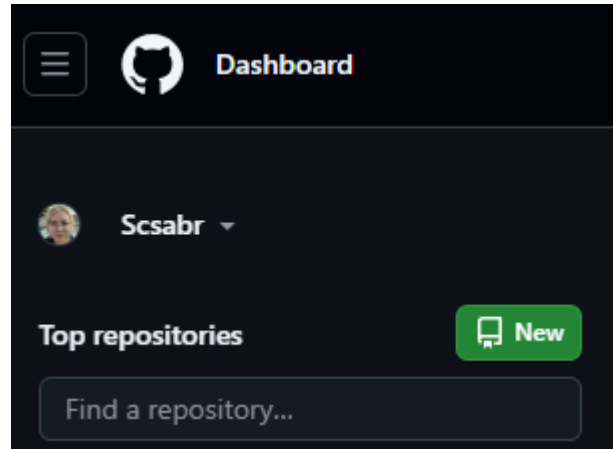
- Log in to Github
 - If you don't already have an account - make one now

<https://github.com/login>

- Sign up for Github Education
 - Gives Pro access to Github

<https://github.com/settings/education/benefits>

Making a Repository



Owner *  Scsabr / Repository name * something-sensible

✔ something-sensible is available.

Great repository names are short and memorable. Need inspiration? How about [scaling-happiness](#) ?

Description (optional)

You don't need to write anything in here but it is sometimes useful

☐ Public
Anyone on the internet can see this repository. You choose who can commit.

☒ Private
You choose who can see and commit to this repository.

Initialize this repository with:

☒ Add a README file
This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: Python

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

License: None

A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

This will set `main` as the default branch. Change the default name in your [settings](#).

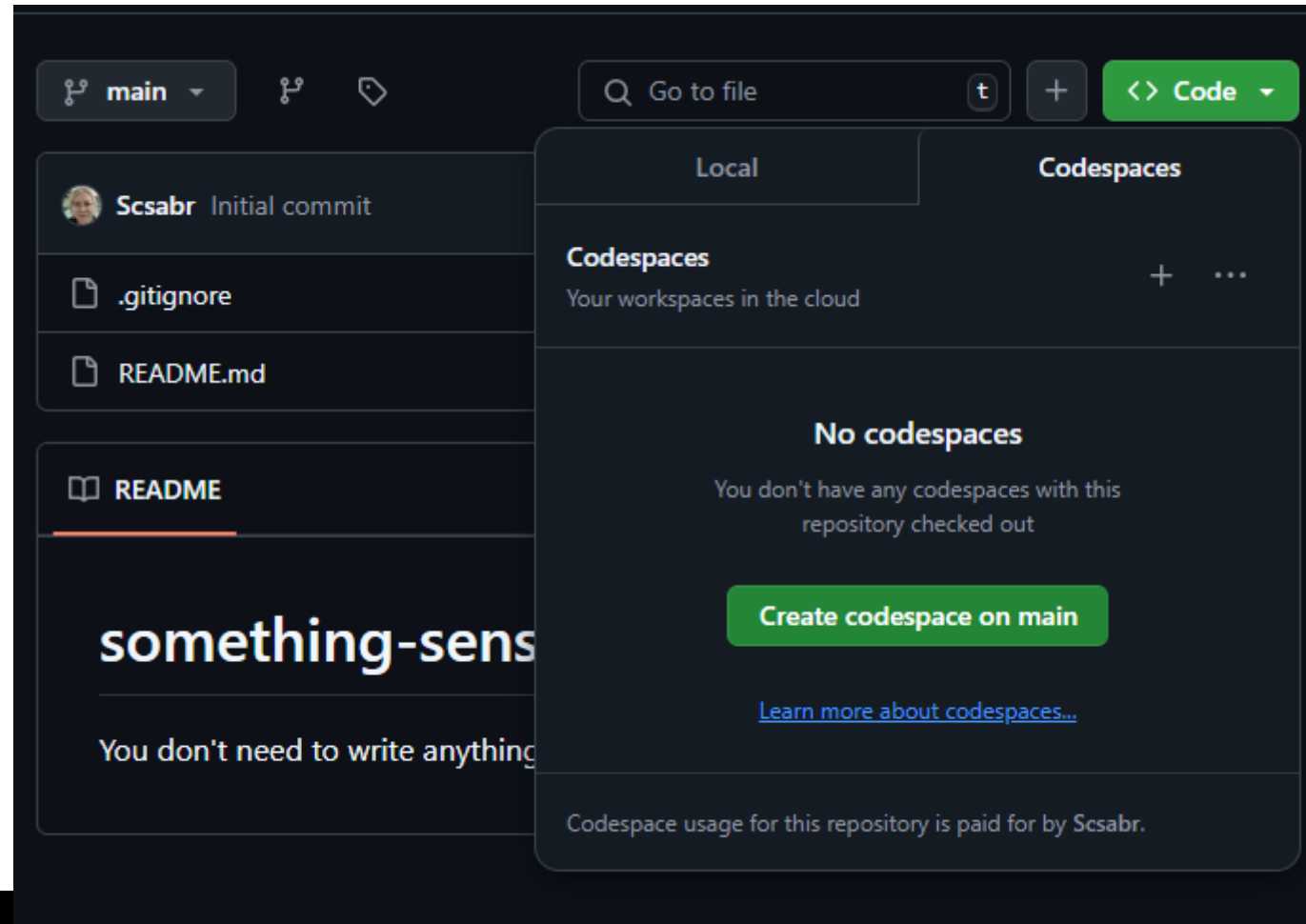
 You are creating a private repository in your personal account.

Create repository

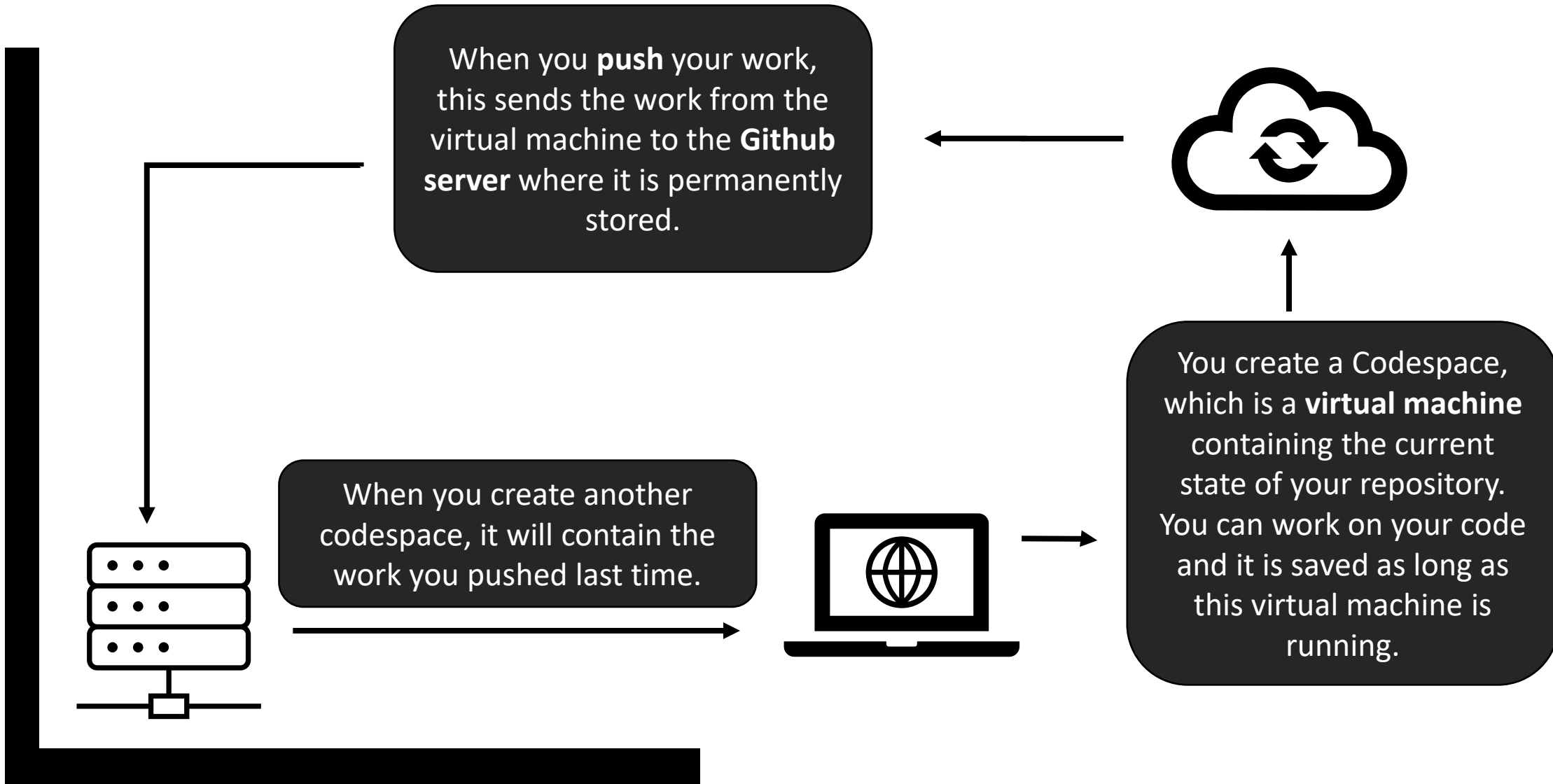
What are these settings?

- Name – what the repo will be called – use a sensible name so you can find it again later
- Description – write a quick description of what's in the repo
- Privacy – always set to private – you can change this later, but it's a good idea to keep your uni work private
- Readme – tick this box.
- Gitignore – optional but recommended – pick whatever language you're using.

Opening a Codespace



What is a Codespace?



What is a Codespace

A Codespace is a linux virtual machine running in the cloud.

- It is created with the current 'main' branch from the Github **repository**.

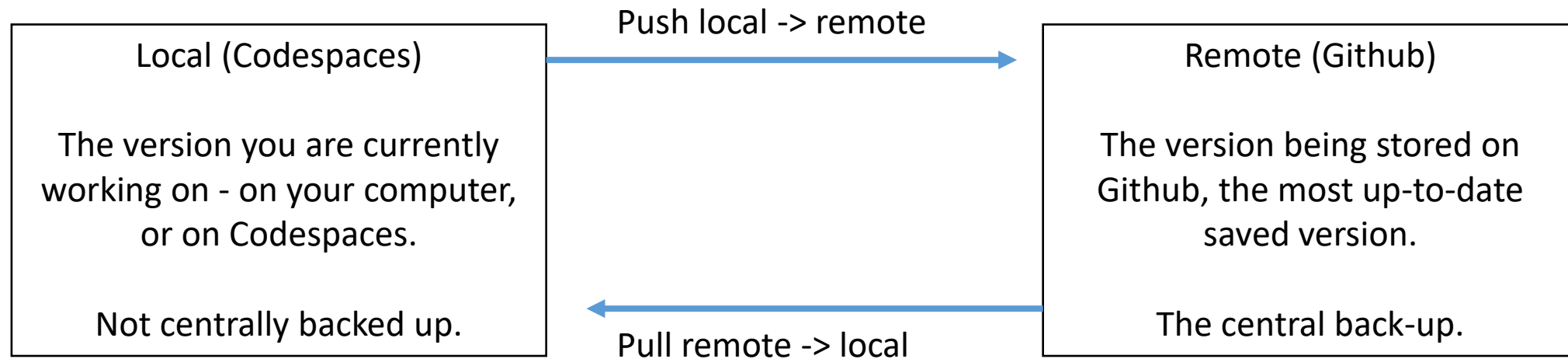
You can run, edit and write new code inside the Codespace

- **however** your work is only saved locally inside the Codespace.

You can **push** your work to save it permanently

- you should do this every session.
- as often as necessary

Using Git: push and pull



Why Codespaces?

Some of you are programming on Windows machines, some on Mac, and some on Linux.

Codespaces means you all work on **the same machines**
– which is helpful to ensure your work is portable

Being able to work on Linux is also essential
– if you work as a developer, it is almost guaranteed that you will use either linux or another Unix-based system (such as a Mac).

Basic Shell Commands

- You will need to be able to **navigate** the linux shell (command line) in Codespaces
- This is very simple, there are just a few commands which you will need to get used to!

`pwd` - tells you your current directory (folder)

`mkdir` - makes a new (empty) named directory, (eg. `mkdir myCode`)

`cd` - move into a named directory (eg. `cd myCode`)

`cd ..` - move up one directory level

`rmdir` - remove an empty named directory, (eg. `rmdir myCode`)

`ls` - list everything in the current directory (files or directories)

`touch` - makes a new (empty) file (eg. `touch myFile.txt`)

`rm` - remove a named file (eg. `rm myFile.txt`)

Task - Setting Up Some Folders

Using the commands you just saw - make these folders and files:

```
.
└─ work
    ├── comp1850
    │   ├── lecture1.md
    │   └── lecture2.md
    ├── comp1860
    │   └── lecture1.md
    ├── comp1870
    │   └── lecture1.md
    └─ timetable.xls
```

Adding, Committing and Pulling.

As we said earlier, the work you have added now is only stored **on the Codespace virtual machine**.

Codespaces **are not permanent** and will close after a few days of inactivity so to keep any changes you need to **save your work** by using some basic git commands.

You can either use the Version Control GUI, or you can use terminal commands – **you should aim to be confident using both**.

In Practice...

Git is very simple to use.



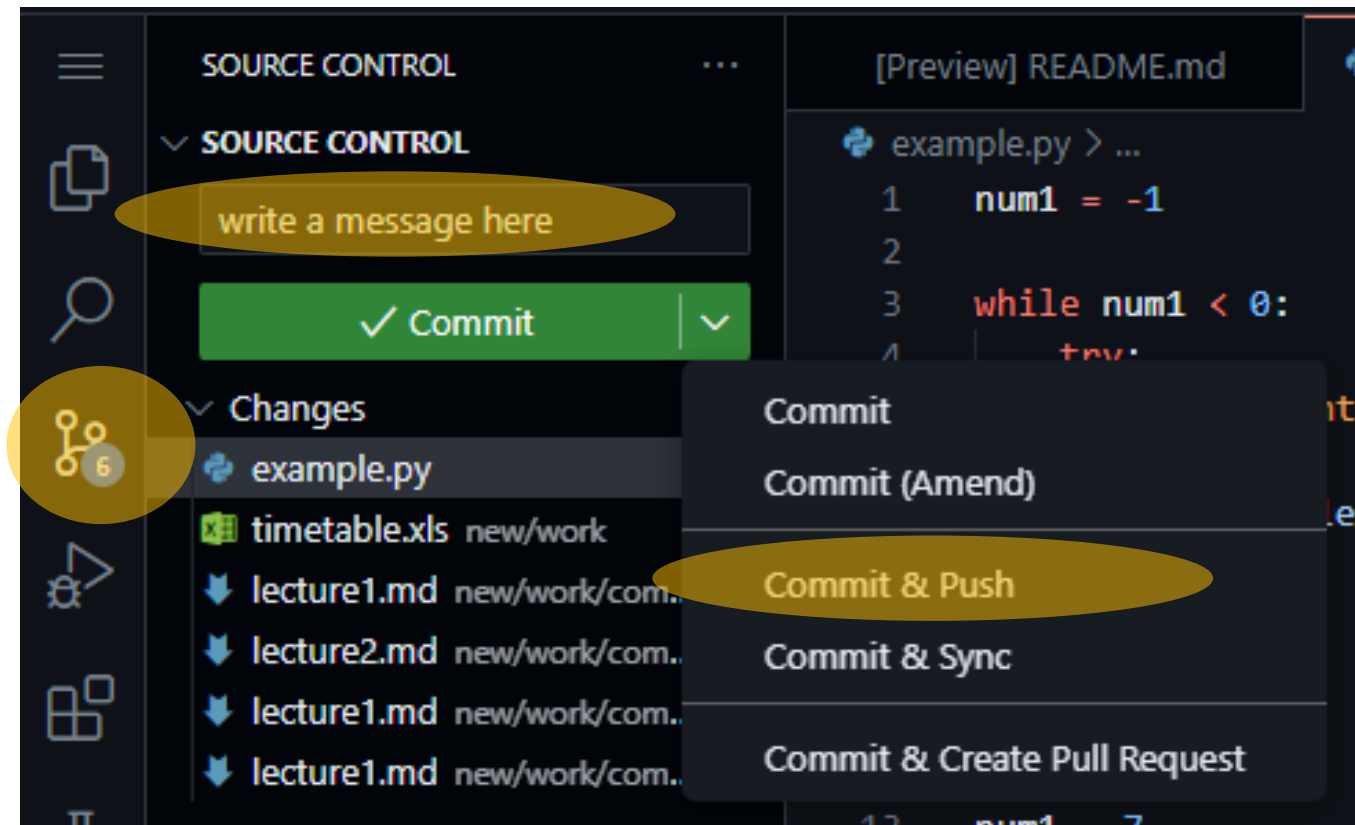
On the command line:

```
git add -A
```

```
git commit -m "message"
```

```
git push
```

→ In the GUI:



What does that mean?

```
git add
```

- **stages** your changes – it tells Git that you have done some work you would like to save.
- A is an **option** which means ‘all’

```
git commit
```

- **bundles** these changes into one ‘commit’ – you can go back to previous commits, so this is basically a way of creating a save point.
- m is an option – it lets you set a commit message. If you don’t add this here, a file will open itself and ask you to write in the commit message.

```
git push
```

- **sends** these changes back to the **remote** Github server (the place where it is stored).

Did it work?

You should check whether the repository has updated on the Github website

- Refresh the browser view of the repository
- Can you see the folders and files you added?

Markdown

Documentation is important on Github

Markdown is a lightweight markup language (like HTML) which is used to write documentation and other basic text documents.

It uses simple ‘tags’ such as # and – to format text.

You will see these used a lot throughout the year
– markdown or “md” files are often used for READMEs.

Claiming your Workspace

All your classwork activities will be stored in **one repository** this semester.

This will mean you are always working in the same place and will have your previous session's work to refer to if needed.

We have prepared this repository for you, and you need to make a copy of it by “forking” our repository

Follow the link and guidance in Semester 1 on Minerva

Task

You are going to make a simple 'about me' markdown page to get used to using markdown.

In /examples there is a document with some different tags as well as a link to the documentation where you will find an even wider range.

Remember to **add, commit and push your work.**

Independent Time

- Every lecture we will try and give you some time to work on your weekly worksheet with support available
- If you are working on the portfolio task - this needs to be **independent** (without getting the answers from someone else)
- You can also use this time to go back through the lecture, ask questions, and get feedback on things from previous weeks!

Worksheet

Each week, the worksheet will be released on Monday.

We do not expect you to be able to complete this on Monday as it will contain material from the whole week.

You should try and complete **all tasks** – the portfolio task is the only one which is formally assessed, but the others are provided to support you to develop your skills.

There will also be **exam-style questions** to help you prepare for the end-of-year exam.

End of Session Routine

Remember – Codespaces aren't permanent and you need to save your work.

```
git add -A
```

```
git commit -m "end of session 1"
```

```
git push
```

Recap:

- Become familiar with the structure and staff for this module
- Understand how you will be assessed this year
- Understand the tools which you will use this year for programming
- Feel confident using Git and Codespaces